CachePilot — Complete Development Plan

Semantic Caching & Cost-Aware Model Routing for RAG

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Delivery approach

Suggested sequence: one phase at a time, with a working main branch at each checkpoint. The time estimates below are planning estimates for one developer and will vary with provider setup and available time.

Phase-by-phase plan

1. Phase 0 — Repository and design

Inspect repository, write architecture decisions, define environment variables, API contracts, data model, threat model, and baseline benchmark.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

2. Phase 1 — Backend foundation

Create FastAPI app, configuration, health endpoints, structured logging, database migrations, and provider interfaces.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

3. Phase 2 — Baseline RAG

Implement ingestion, chunking, embeddings, vector retrieval, grounded generation, citations, and a reproducible baseline.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

4. Phase 3 — Semantic cache

Implement scoped keys, similarity lookup, threshold policy, TTL, invalidation, bypass, and cache metrics.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

5. Phase 4 — Routing and usage ledger

Implement routing policy, model adapters, fallback handling, token/cost accounting, and per-request telemetry.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

6. Phase 5 — Dashboard

Build React dashboard for cost, latency, routing, cache, errors, and evaluation results.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

7. Phase 6 — Evaluation and hardening

Run benchmarks, test adversarial cache cases, add rate limits, validate secrets handling, and document limitations.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

8. Phase 7 — Packaging

Docker Compose, seed dataset, .env.example, API guide, architecture diagram, demo script, and reproducible benchmark report.

Exit gate: implementation runs locally, relevant tests pass, README/docs are updated, and Antigravity reports changed files, commands executed, test results, and unresolved issues.

Suggested schedule

- Week 1: Phase 0–1, architecture decisions, repo setup, database, API skeleton, and local environment.
- Week 2: Core data pipeline and baseline end-to-end workflow.
- Week 3: Project-specific core capability and failure handling.
- Week 4: Frontend dashboard/inspection UI and telemetry.
- Week 5: Security tests, evaluation benchmark, tuning, and documentation.
- Week 6: Packaging, demo data, final QA, screenshots, architecture diagram, and portfolio write-up.

Milestones and evidence

- M1 — Architecture approved: diagrams, data model, API contracts, security assumptions, and decision log.
- M2 — Vertical slice: one document ingested and one question answered end to end.
- M3 — Core differentiator: feature implemented with unit and integration tests.
- M4 — Observable application: dashboard/trace shows relevant metrics for each request.
- M5 — Evaluation: baseline and enhanced system measured on the same versioned dataset.
- M6 — Release candidate: automated tests pass, Docker startup is reproducible, documentation complete.

Risk register

- Similarity thresholds can return a semantically related but incorrect answer; require evaluation and conservative defaults.
- Cached answers can become stale; version keys, TTL, and explicit invalidation are mandatory.
- Cost figures may be estimates if providers do not expose authoritative billing; show assumptions and units.
- Routing can lower quality; retain a high-quality fallback and evaluate by question category.

Release checklist

- Clean clone and setup verified from README.
- No committed credentials, local database files, or user data.
- Migrations, seed script, lint, type checks, tests, and production frontend build pass.
- Critical security boundaries have explicit tests.
- Evaluation report includes dataset description, method, baseline, results, limitations, and reproducibility details.
- Demo does not expose real secrets or sensitive documents.

- Resume claims are updated only after measured results exist; never invent percentages or claim unverified improvements.